

Arduino LoRaWAN MAC на языке C (LMIC)

Информация о продукте LMIC

Библиотека LMIC изначально была разработана и продавалась исследовательской лабораторией IBM Zurich (IBM Research GmbH), 8803 Rüschlikon, Switzerland. Для получения дополнительной информации обращайтесь: lrsc@zurich.ibm.com. Этот документ взят из версии V1.6 от июля 2016 года.

Библиотека была адаптирована для Arduino Матейсом Коойманом и Томасом Телкампом. Эта версия документа описывает версию, поддерживаемую MCCI Corporation по адресу <https://github.com/mccicatenar/arduino-lmic/>.

© 2018-2024 Корпорация MCCI

Copyright MCCI Corporation, 2018-2024. Все права защищены. Распространяется на условиях файла LICENSE, который находится по адресу <https://github.com/mcci-catena/arduino-lmic/blob/master/LICENSE>.

© 2014-2016 Корпорация IBM

Авторские права принадлежат International Business Machines Corporation, 2014-2016. Все права защищены.

Ниже перечислены товарные знаки или зарегистрированные товарные знаки International Business Machines Corporation в США и/или других странах: IBM, логотип IBM, Ready for IBM Technology.

MCCI и MCCI Catena являются зарегистрированными товарными знаками MCCI Corporation.

LoRa является зарегистрированной торговой маркой Semtech Corporation.

LoRaWAN является зарегистрированной торговой маркой LoRa Alliance.

Другие названия компаний, продуктов и услуг могут быть товарными знаками или знаками обслуживания других лиц.

Вся информация, содержащаяся в этом документе, может быть изменена без предварительного уведомления. Информация, содержащаяся в этом документе, не влияет и не изменяет спецификации или гарантии продуктов IBM. Ничто в этом документе не должно действовать как явная или подразумеваемая лицензия или возмещение в соответствии с правами интеллектуальной собственности IBM или третьих лиц. Вся информация, содержащаяся в этом документе, была получена в определенных средах и представлена в качестве иллюстрации. Результаты, полученные в других рабочих средах, могут отличаться. ИНФОРМАЦИЯ, СОДЕРЖАЩАЯСЯ В ЭТОМ ДОКУМЕНТЕ, ПРЕДОСТАВЛЯЕТСЯ НА УСЛОВИЯХ «КАК ЕСТЬ»

ОСНОВАНИЕ. Ни при каких обстоятельствах IBM не будет нести ответственности за ущерб, возникший прямо или косвенно в результате любого использования информации, содержащейся в этом документе.

Оглавление

1.	Введение.....	5	1.1 Поддерживаемые версии и функции LoRaWAN.....	6	1.2 Поддержка классов А и В.....	6						
2.	Модель программирования и API.....	7	2.1 Модель программирования.....	7	2.2 Функции времени выполнения.....	9	2.3 Обратные вызовы приложения.....	10	2.4 Структура LMIC.....	11	2.5 Функции API.....	12
3.	Уровень абстракции оборудования	22	3.1 Интерфейс HAL	23	3.2 Реализация эталонного HAL для Arduino	25						
4.	Примеры.....	26										
5.	История выпусков.....	27	5.1 История выпусков IBM.....	27								

1. Введение

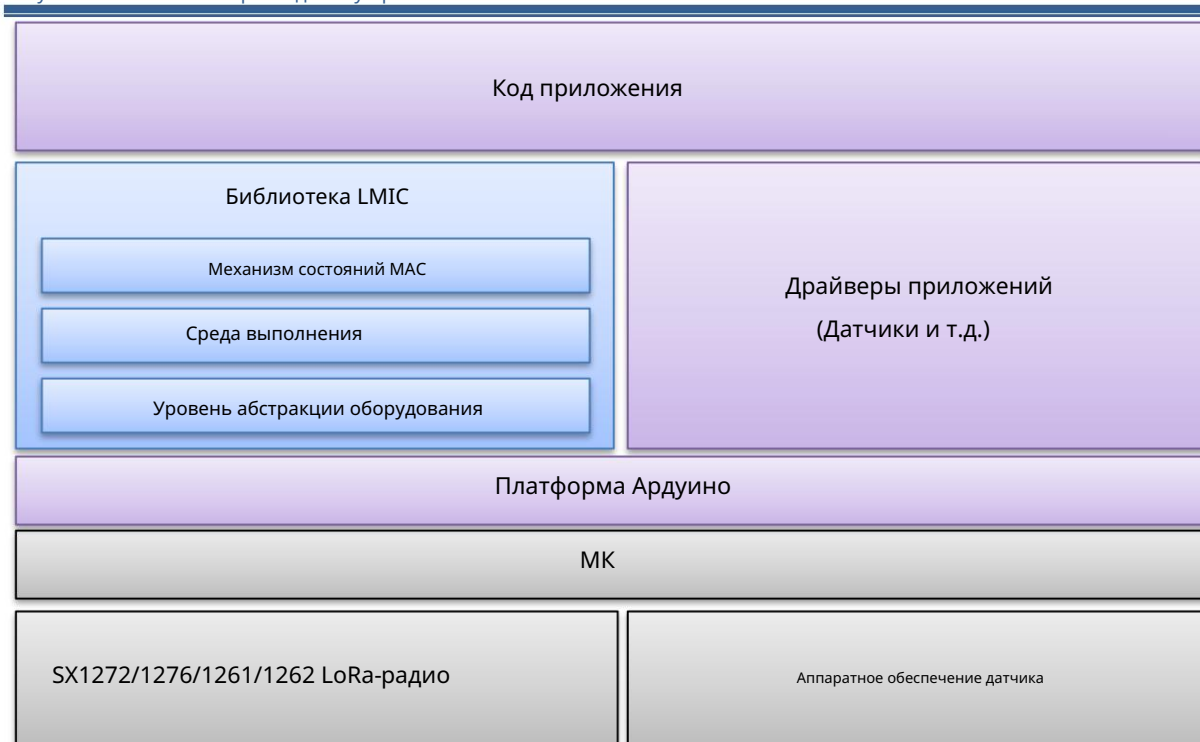
Библиотека Arduino IBM LoRaWAN C (LMIC) является переносимой реализацией спецификации конечного устройства LoRaWAN® 1.0.3 на языке программирования C. («LMIC» означает «LoRaWAN MAC на языке C»). Она поддерживает варианты спецификации EU-868, US-915, AU-915, AS-923, KR-920 и IN-866 и может работать с устройствами классов A и B. Библиотека заботится обо всех логических состояниях MAC и временных ограничениях и управляет радиоустройствами SEMTECH SX1272, SX1276, SX1261 или SX1262. Таким образом,

приложения могут свободно выполнять другие задачи, а соответствие протоколу гарантируется библиотекой. Чтобы обеспечить соответствие спецификации и связанным с ней правилам, механизм состояний был протестирован и проверен с использованием среды логического моделирования. Библиотека была тщательно спроектирована для точного соответствия временным ограничениям протокола MAC и даже для учета возможных отклонений часов в расчетах времени. Приложения могут получать доступ ко всем функциям и настраивать их с помощью простой модели программирования на основе событий и не должны иметь дело со специфическими для платформы деталями, такими как обработчики прерываний. Используя тонкий уровень абстракции оборудования (HAL), библиотеку можно легко переносить на новые аппаратные платформы. Предоставляется Arduino HAL, который позволяет легко интегрировать ее с большинством вариантов Arduino.

Поддерживаются как 8-битные, так и 32-битные платформы AVR.

В дополнение к предоставленной библиотеке LMIC, реальному приложению также нужны драйверы для датчиков или другого оборудования, которым оно хочет управлять. Эти драйверы приложений выходят за рамки этого документа и не являются частью этой библиотеки.

Рисунок 1. Компоненты прикладного устройства



Высокоуровневое представление всех компонентов устройства приложения.

1.1 Поддерживаемые версии и функции LoRaWAN

Библиотека LMIC поддерживает работу класса A, как описано в спецификации LoRaWAN V1.0.3 и V1.0.2. Она не поддерживает V1.1.

Код поддержки класса B предоставляется, но не тестировался.

Многоадресные нисходящие каналы LoRaWAN 1.0.3 класса A не поддерживаются.

Работа класса C не поддерживается.

Библиотека была протестирована на соответствие следующим спецификациям теста LoRaWAN 1.0.2, реализованным RedwoodComm в их тестере RWC5020A с использованием прошивки версии 1.170. В качестве эталонного устройства использовалось MCCI Catena 4610 (с радио SX1276).

- EU868 V1.5 (без дополнительных скоростей передачи данных)
- US915 V1.3 (исключая дополнительные скорости передачи данных; тестирование проводилось с использованием каналов 0–7 и 64). Тест TxPower не пройден, поскольку LMIC соответствует LoRaWAN V1.0.3. Тест TxPower использует значение, определенное для V1.0.3, но не для V1.0.2, и ожидает, что устройство отклонит это значение.
- AS923 V1.1 (без дополнительных скоростей передачи данных)
- KR920 V1.2
- IN865 V1.0

Все испытания проводились с использованием проводного соединения.

1.2 Поддержка классов A и B

Библиотеку Arduino LMIC можно настроить для поддержки работы LoRaWAN классов A и B. Устройство класса A принимает только в фиксированные моменты времени после передачи сообщения. Это позволяет работать с низким энергопотреблением, но означает, что задержка нисходящей линии связи контролируется частотой передачи устройством. Устройство класса B синхронизируется с маяками, передаваемыми сетью, и прослушивает сообщения с определенными интервалами («слоты ping») в течение интервала между маяками.

Устройства (и библиотека LMIC) изначально являются устройствами класса A и переключаются на класс B на основе запросов от прикладного уровня устройства.

В этом документе термин «pinging» используется для обозначения того, что LMIC работает в классе B, а также прослушивает нисходящий канал во время слотов ping. Если устройство pinging, то LMIC также должен отслеживать маяк. Можно отслеживать маяк (возможно, для целей синхронизации времени) без включения pinging.

Поскольку многие устройства и сети поддерживают только операции класса A, библиотеку можно настроить во время компиляции, чтобы исключить поддержку отслеживания и пингования. Можно исключить поддержку пингования, не исключая поддержку отслеживания, но это не проверенная конфигурация.

2. Модель программирования и API

Библиотека LMIC может быть доступна через набор функций API, функции времени выполнения, функции обратного вызова и глобальную структуру данных LMIC. Интерфейс определен в одном заголовочном файле «lmic.h», который должны включать все приложения.

```
#включить "lmic.h"
```

Версия библиотеки соответствует Semantic Versioning 2.0.0 (<https://semver.org/>). Один символ представляет версию. Биты 31..24 представляют основную версию, биты 23..15 — второстепенную версию, а биты 15..8 представляют патч. Биты 7..0 используются для представления предварительной версии, называемой «ЛОКАЛЬНОЙ» для исторической причины.

Для построения номеров версий используется функциональный макрос ARDUINO_LMIC_VERSION_CALC() .

```
#определить ARDUINO_LMIC_VERSION ARDUINO_LMIC_VERSION_CALC(2, 3, 2, 0)
```

Для удобства библиотека предоставляет макросы-функции ARDUINO_LMIC_VERSION_GET_MAJOR(), ARDUINO_LMIC_VERSION_GET_MINOR(), ARDUINO_LMIC_VERSION_GET_PATCH() и ARDUINO_LMIC_VERSION_GET_LOCAL(), который извлекает соответствующее поле из номера версии.

Номера версий представлены примитивно — четыре поля просто упакованы в 32-битное слово. Это делает их простыми для отображения, но сложными для сравнения. Предварительные версии пронумерованы выше, чем релизы, но сравниваются реже. Для упрощения сравнения номеров версий предусмотрено несколько макросов. ARDUINO_LMIC_VERSION_TO_ORDINAL() с помощью ARDUINO_LMIC_VERSION_COMPARE_LT(v1, v2) возвращает число, которое можно сравнить с помощью обычных операторов сравнения C. ARDUINO_LMIC_VERSION_COMPARE_LT(v1, v2) сравнивает два номера версий и возвращает ненулевое значение, если v1 меньше v2 (после преобразования обоих в порядковые числа). ARDUINO_LMIC_VERSION_COMPARE_LE(v1, v2), ARDUINO_LMIC_VERSION_COMPARE_GT(v1, v2) и ARDUINO_LMIC_VERSION_COMPARE_GE(v1, v2) проверяют на наличие отношений «меньше или равно», «больше или равно».

Для идентификации исходной версии библиотеки IBM LMIC в этом заголовочном файле определены две константы.

```
#define LMIC_VERSION_MAJOR 1
#define LMIC_VERSION_MINOR 6
```

Эти строки версии идентифицируют базовую версию библиотеки и не изменяются.

2.1 Модель программирования

Библиотека LMIC предлагает простую модель программирования на основе событий, где все события протокола отправляются в функцию обратного вызова onEvent() приложения (см. 2.3.4). Чтобы освободить приложение от таких деталей, как тайминги или прерывания, библиотека имеет встроенную среду выполнения, которая заботится об очередях таймеров и управлении заданиями.

2.1.1 Прикладные задачи

В этой модели весь код приложения выполняется в так называемых заданиях, которые выполняются в главном потоке функцией планировщика времени выполнения os_runloop() (см. 2.2.6). Эти задания приложения кодируются как обычные функции C и могут управляться с помощью функций времени выполнения, описанных в разделе 2.1.3. Для управления заданиями требуется дополнительная структура управления для каждого задания osjob_t, которая идентифицирует задание и сохраняет

Контекстная информация. Задания не должны быть длительными, чтобы обеспечить бесперебойную работу! Они следует только обновлять состояние и планировать действия, которые вызовут новые обратные вызовы заданий или событий.

2.1.2 Основной цикл событий

Все, что должно сделать приложение, это инициализировать среду выполнения с помощью `os_init()` или `os_init_ex()`. функция, а затем периодически вызывать функцию планировщика заданий `os_runloop_once()`. Для загрузки действий протокола и генерации событий необходимо настроить начальное задание. Поэтому задание запуска планируется с помощью функции `os_setCallback()`.

```
osjob_t initjob;

// недействительная настройка {} {
    // инициализация среды выполнения
    os_init();
    // настройка начального задания
    os_setCallback(&initjob, initfunc);
}

// пустой цикл {} {
    // выполнение запланированных заданий и событий
    os_runloop_once();
}
```

Код запуска, показанный в функции `initfunc()` ниже, инициализирует MAC и начинает подключение к сети.

```
// начальная работа
static void initfunc (osjob_t* j) {
    // сбросить состояние MAC
    LMIC_reset();
    // начать присоединение
    LMIC_startjoining();
    // инициализация выполнена - будет вызван обратный вызов onEvent()...
}
```

Функция `initfunc()` вернется немедленно, а функция обратного вызова `onEvent()` будет вызвана планировщиком позже для событий `EV_JOINING`, `EV_JOINED` или `EV_JOIN_FAILED`.

2.1.3 Время ОС

LMIC использует значения типа `ostime_t` для представления времени в тиках. Скорость этих тиков по умолчанию составляет 32768 тиков в секунду, но может быть настроена во время компиляции на любое значение от 10000 тиков в секунду до 64516 тиков в секунду.

В общем случае один тик не является целым числом микросекунд или миллисекунд. Для переключения вперед и назад предусмотрены удобные функции.

```
typedef int32_t ostime_t;
```

Обратите внимание, что это знаковое целое число ; необходимо соблюдать осторожность при вычислении разностей, чтобы избежать переполнения. Время ОС начинается с нуля и равномерно увеличивается до `INT32_MAX`; затем оно переносится на `INT32_MIN` и равномерно увеличивается до нуля, и повторяется. Вместо того, чтобы сравнивать два `ostime_t` рекомендуемая практика — вычесть их и посмотреть, является ли результат положительным или отрицательным.

2.2 Функции времени выполнения

Функции выполнения, упомянутые выше, используются для управления средой выполнения. Это включает в себя инициализация, планирование и выполнение заданий времени выполнения.

2.2.1 void os_init ()

Инициализируйте операционную систему, вызвав os_init_ex(NULL).

2.2.2 void os_init_ex (const void * pHalData)

Для облегчения использования этой библиотеки на нескольких платформах процедура os_init_ex() принимает произвольный указатель на данные платформы. Реализация HAL по умолчанию для Arduino LMIC ожидает, что этот указатель будет ссылкой на объект C++ struct Imic_pinmap. Для получения дополнительной информации см. README.md.

2.2.3 void os_setCallback (osjob_t* задание, osjobcb_t cb)

Подготовить немедленно запускаемую задачу. Эту функцию можно вызвать в любое время, в том числе из контекстов обработчика прерываний (например, если стало доступно новое значение датчика).

2.2.4 void os_setTimedCallback (osjob_t* задание, ostime_t время, osjobcb_t cb)

Запланировать выполнение задания по времени в указанную временную метку (абсолютное системное время). Эту функцию можно вызвать в любое время, в том числе из контекстов обработчика прерываний.

2.2.5 void os_clearCallback (osjob_t* задание)

Отменить задание времени выполнения. Ранее запланированное задание времени выполнения удаляется из очередей таймера и выполнения. Задание идентифицируется адресом структуры задания. Функция не оказывает никакого эффекта, если указанное задание еще не запланировано.

2.2.6 void os_runloop ()

Выполнять задания времени выполнения из таймера и из очередей выполнения. Эта функция является основным диспетчером действий. Она не возвращает значение и должна быть запущена в основном потоке. Эта процедура обычно не используется в средах Arduino, так как она отключает нормальный вызов функции Arduino loop().

2.2.7 void os_runloop_once ()

Выполнять задания времени выполнения из таймера и из очередей выполнения. Эта функция похожа на os_runloop(), за исключением того, что она возвращается после отправки первого доступного задания.

2.2.8 ostime_t os_getTime ()

Запрос абсолютного системного времени (в тиках).

2.2.9 ostime_t us2osticks(s4_t us)

Возвращает такты, соответствующие целочисленному значению us. Это может быть макрос, подобный функции, поэтому us может быть оценен более одного раза. Любая дробная часть вычисления отбрасывается.

2.2.10 `ostime_t us2osticksCeil(s4_t us)`

Возвращает такты, соответствующие целочисленному значению `us`. Это может быть макрос, подобный функции, поэтому `us` может быть оценен более одного раза. Если дробная часть вычисления не равна нулю, результат увеличивается до положительной бесконечности.

2.2.11 `ostime_t us2osticksRound(s4_t us)`

Возвращает такты, соответствующие целочисленному значению `us`. Это может быть макрос, подобный функции, поэтому `us` может быть оценен более одного раза. Результат округляется до ближайшего такта.

2.2.12 `ostime_t ms2osticks(s4_t ms)`

Возвращает тики, соответствующие целочисленному значению миллисекунды `ms`. Это может быть макрос, подобный функции, поэтому `ms` может быть оценено более одного раза. Если дробная часть вычисления не равна нулю, результат увеличивается до положительной бесконечности.

2.2.13 `ostime_t ms2osticksCeil(s4_t ms)`

Возвращает тики, соответствующие целочисленному значению миллисекунды `ms`. Это может быть макрос, подобный функции, поэтому `ms` может быть оценено более одного раза.

2.2.14 `ostime_t ms2osticksRound(s4_t ms)`

Возвращает тики, соответствующие целочисленному значению миллисекунды `ms`. Это может быть макрос, подобный функции, поэтому `ms` может быть оценено более одного раза. Результат округляется до ближайшего тика.

2.2.15 `ostime_t sec2osticks(s4_t sec)`

Возвращает такты, соответствующие значению секунды целого числа `sec`. Это может быть макрос, подобный функции, поэтому `sec` может быть оценен более одного раза.

2.2.16 `S4_t osticks2ms(ostime_t os)`

Возвращает миллисекунды, соответствующие значению тика `os`. Это может быть макрос, подобный функции, поэтому `os` может быть оценен более одного раза.

2.2.17 `S4_t osticks2us(ostime_t os)`

Возвращает микросекунды, соответствующие значению тика `os`. Это может быть макрос, подобный функции, поэтому `os` может быть оценен более одного раза.

2.3 Обратные вызовы приложений

Библиотека LMIC требует, чтобы приложение реализовало несколько функций обратного вызова. Эти функции вызываются механизмом состояний для запроса информации, специфичной для приложения, и для доставки событий состояния в приложение.

Urcalls по имени из LMIC в код приложения устарели и будут удалены в будущих версиях LMIC. API-интерфейсы предоставления (`os_getDevEui`, `os_getDevKey` и `os_getArtEui`) будут заменены API-интерфейсами защищенных элементов в версии 4. API-интерфейс `onEvent` будет отключен по умолчанию в версии 4 и удален в версии 5.

2.3.1 void os_getDevEui (u1_t* buf)

Реализация этой функции обратного вызова должна предоставить EUI устройства и скопировать его в указанный буфер. Длина EUI устройства составляет 8 байтов, и он хранится в формате little-endian, то есть сначала младший байт (LSBF).

2.3.2 void os_getDevKey (u1_t* buf)

Реализация этой функции обратного вызова должна предоставить криптографический ключ приложения, специфичный для устройства, и скопировать его в указанный буфер. Ключ приложения, специфичный для устройства, представляет собой 128-битный ключ AES (длиной 16 байт).

2.3.3 void os_getArtEui (u1_t* buf)

Реализация этой функции обратного вызова должна предоставить EUI приложения и скопировать его в указанный буфер. EUI приложения имеет длину 8 байт и хранится в формате little-endian, то есть, наименее значимый байт первым (LSBF).

2.3.4 void onEvent (ev_t ev)

Эта функция, если она предоставлена, вызывается для сообщения о событиях LMIC (таких как завершение передачи или получение сообщения по нисходящей линии связи). Это устаревшая функция. В V3, если это имя конфликтует с именем в вашем приложении, вы можете отключить использование этого имени, поместив следующее в ваш файл конфигурации:

```
#define LMIC_ENABLE_onEvent 0
```

Urcalls по имени из LMIC в код приложения устарели и будут удалены в будущих версиях LMIC. Доступна подходящая замена. См. обсуждение

LMIC_registerEventCb(), ниже.

2.4 Структура стран с низким и средним уровнем дохода

Вместо передачи многочисленных параметров туда и обратно между API и функциями обратного вызова, информация о состоянии протокола может быть доступна через глобальную структуру LMIC, как показано ниже. Все поля, кроме явно указанных ниже, доступны только для чтения и не должны изменяться.

```
структура lmic_t {
    u1_t      кадр[MAX_LEN_FRAME];
    u1_t      dataLen; // 0 нет данных или данные нулевой длины, >0 байт данных
    u1_t      dataRequest; // 0 или начало данных (dataBeg-1 — это порт)

    u1_t      txCnt;
    u1_t      txrxFlags; // флаги транзакции (комбинация TX-RX)

    u1_t      pendTxPort;
    u1_t      pendTxConf; // подтвержденные данные
    u1_t      pendTxLen;
    u1_t      pendTxData[MAX_LEN_PAYLOAD];

    u1_t      bcnChnl;
    u1_t      bcnRxsyms;
    ostime_t  bcnRxtime;
    bcninfo_t bcninfo; // Последняя полученная информация о маяке
```

```
...
...
};
```

В этом документе не описывается вся структура подробно, поскольку большинство полей структуры LMIC используется только внутри. Наиболее важными полями для проверки при приеме (событие EV_RXCOMPLETE или EV_TXCOMPLETE) являются txrxFlags для информации о состоянии и frame[] и dataLen / dataBeg для полученных данных полезной нагрузки приложения. Для передачи данных наиболее важными полями являются pendTxData[], pendTxLen, pendTxPort и pendTxConf, которые используются в качестве входных данных для функции API LMIC_setTxData() (см. 2.5.13).

Для событий EV_RXCOMPLETE и EV_TXCOMPLETE следует оценить поле txrxFlags. Определены следующие флаги:

- TXRX_ACK: подтверждено, что кадр UP был подтвержден (взаимоисключающее с TXRX_NACK)
- TXRX_NACK: подтверждено, что кадр UP не был подтвержден (взаимоисключающее с TXRX_ACK)
- TXRX_PORT: байт порта содержится в полученном кадре со смещением LMIC.dataBeg – 1.
- TXRX_NOPORT: байт порта недоступен.
- TXRX_DNW1: получено в первом слоте DOWN (взаимоисключающее с TXRX_DNW2)
- TXRX_DNW2: получено во втором слоте DOWN (взаимоисключающее с TXRX_DNW1)
- TXRX_PING: получено в запланированном слоте RX
- TXRX_LENERR: переданное сообщение было отклонено, поскольку оно было длиннее, чем установленная скорость передачи данных.

Для события EV_TXCOMPLETE поля имеют следующие значения:

Полученный рамка	LMIC.txrxFlags								LMIC.dataLe н	LMIC.dataBe г
	АК К	НАК К	К Т	НОПОР Т	DNW 1	DNW 2	г П	НАКЛОНРЕТСЯ Р		
ничего	0	0	0	1	0	0	0	0	0	0
пустой фрейм xx порт			0	1	x	x	0	0	0	x
только	xx		1	0	x	x	0	0	0	x
порт+полезная нагрузка г	xx		1	0	x	x	0	0	x	x
Сообщение не получено, передавать сообщение слишком долго	0	0	0	1	x	x	x	1	0	0

Для события EV_RXCOMPLETE поля имеют следующие значения:

Полученный рамка	LMIC.txrxFlags								LMIC.dataLen	LMIC.dataBeg
	ACK	NACK	PORT	NOPORT	DNW1	DNW2	PING			
пустой порт кадра	0	0	0	1	0	0	1		0	x
только	0	0	1	0	0	0	1		0	x
порт+полезная нагрузка	0	0	1	0	0	0	1		x	x

2.5 Функции API

Библиотека LMIC предлагает набор функций API для управления состоянием MAC и запуска действий протокола.

2.5.1 недействительный LMIC_reset ()

Сбросьте состояние MAC. Сеанс и ожидающие передачи данных будут отменены.

2.5.2 bit_t LMIC_startJoining ()

Немедленно начать присоединение к сети. Будет вызвано неявно другими функциями API, если сеанс еще не установлен. Будут сгенерированы события EV_JOINING и EV_JOINED или EV_JOIN_FAILED.

2.5.3 недействительный LMIC_tryRejoin ()

Проверьте, нет ли поблизости других сетей, к которым можно присоединиться. Сеанс с текущей сетью сохраняется, если не найдена новая сеть. Будут сгенерированы события EV_JOINED или EV_REJOIN_FAILED.

2.5.4 void LMIC_setSession (u4_t netid, devaddr_t devaddr, u1_t* nwkey, u1_t* artkey)

Установите статические параметры сеанса. Вместо динамического установления сеанса путем присоединения к сети можно предоставить предварительно вычисленные параметры сеанса. Для возобновления сеанса с предварительно вычисленными параметрами счетчики последовательности кадров (LMIC.seqnoUp и LMIC.seqnoDn) должны быть восстановлены до последних значений ценности.

2.5.5 bit_t LMIC_setupBand (u1_t bandidx, s1_t txpow, u2_t txcap)

Инициализирует определенный диапазон рабочего цикла с указанными свойствами мощности передачи и рабочего цикла (1/txcap). Возвращает ненулевое значение в случае успеха, ноль в случае неудачи. Некоторые регионы не позволяют определять диапазоны рабочего цикла, и в этом случае эта функция всегда будет возвращать ноль.

Аргумент bandidx выбирает диапазон; он должен быть в диапазоне 0 ≤ bandidx < MAX_BANDS. Для регионов, подобных ЕС, определены следующие названия диапазонов: BAND_MILLI, предназначенный для использования на каналах с рабочим циклом 0,1%; BAND_CENTI, предназначенный для использования на каналах с рабочим циклом 1%; и BAND_DECI, предназначенный для использования на каналах с рабочим циклом 10%. Четвертый диапазон, BAND_AUX, доступен для использования пользовательским кодом.

2.5.6 bit_t LMIC_setupChannel (канал u1_t , частота u4_t , карта drmap u2_t , диапазон s1_t)

Создайте новый канал или измените существующий. Если freq не равен нулю, канал включен; в противном случае он отключен. Аргумент drmap — это битовая карта, указывающая, какие скорости передачи данных (от 0 до 15) включены для этого канала. Band, если он находится в диапазоне 0..3, назначает канал указанному диапазону рабочего цикла — значение этого параметра зависит от региона. Если установлено значение -1, то соответствующий диапазон рабочего цикла выбирается на основе значения freq.

Результат ненулевой, если канал был успешно настроен; в противном случае — ноль.

Для любого региона есть каналы, которые нельзя изменить («каналы по умолчанию»). Вы не можете отключить или изменить их, но попытка установить канал по умолчанию на его значение по умолчанию не является ошибкой.

В некоторых регионах (например, US915 или AU915) настройка каналов невозможна.

2.5.7 void LMIC_disableChannel (канал u1_t)

Отключить указанный канал. Каналы по умолчанию не могут быть отключены.

2.5.8 u1_t LMIC_queryNumDefaultChannels ()

Верните количество каналов по умолчанию, определенных этим регионом. В регионах EU868, KR920 и IN866 результат равен 3; в регионе AS923 результат равен 2; в регионах US и AU результат равен 72.

2.5.9 void LMIC_setAdrMode (bit_t включен)

Включить или отключить адаптацию скорости передачи данных. Следует отключить, если устройство мобильное.

2.5.10 void LMIC_setLinkCheckMode (bit_t включен)

Включить/отключить проверку ссылок. Режим проверки ссылок включен по умолчанию и используется для периодической проверки сетевого подключения. Должен вызываться только в случае установления сеанса.

2.5.11 void LMIC_setDrTxpow (dr_t dr, s1_t txpow)

Установите скорость передачи данных и мощность передачи. Следует использовать только если отключена адаптация скорости передачи данных.

2.5.12 bit_t LMIC_queryTxReady ()

Возвращает ненулевое значение, если LMIC готов принять передаваемые данные, и ноль, если попытка передачи будет отклонена.

2.5.13 ~~недействительный~~ LMIC_setTxData ()

Подготовить передачу данных вверх по течению в ближайшее возможное время. Предполагается, что `pendTxData`, `pendTxLen`, `pendTxPort` и `pendTxConf` уже установлены. Данные длиной `LMIC.pendTxLen` из массива `LMIC.pendTxData[]` будут отправлены на порт `LMIC.pendTxPort`. Если `LMIC.pendTxConf` имеет значение `true`, будет запрошено подтверждение сервера. Событие `EV_TXCOMPLETE` будет сгенерировано после завершения транзакции, т. е. после отправки данных и получения возможных данных down или запрошенного подтверждения.

Если включена адаптация скорости передачи данных, эта функция проверит, возможно ли передать сообщение с текущей скоростью передачи данных. Если нет, скорость передачи данных будет увеличена, если это возможно. Это необходимо для обеспечения максимальной совместимости программного обеспечения со старыми приложениями, которые не понимают побочных эффектов изменения скорости для более длинных сообщений. Однако не всегда гарантируется возможность отправки сообщения заданного размера, из-за ограничений регионального плана и текущих требований к работе сети (как предусмотрено нисходящими линиями связи). До версии 3.2 LMIC в любом случае передавал бы данные; с версии 3.2 он будет сообщать об ошибке. Таким образом, возможно, потребуется внести изменения в приложения.

Из-за многочисленных постусловий, которые необходимо проверить после вызова `LMIC_setTxData`, мы настоятельно рекомендуем использовать вместо этого `LMIC_setTxData2()`.

2.5.14 ~~недействительный~~ LMIC_setTxData_strict ()

Подготовить передачу данных вверх по течению в ближайшее возможное время. Вызывающий должен сначала инициализировать `LMIC.pendTxData[]`, `LMIC.pendTxLen`, `LMIC.pendTxPort` и `LMIC.pendTxConf`. Данные длины `LMIC.pendTxLen` из массива `LMIC.pendTxData[]` будут отправлены на порт `LMIC.pendTxPort`. Если `LMIC.pendTxConf` имеет значение `true`, будет запрошено подтверждение от сетевого сервера. Событие `EV_TXCOMPLETE` будет сгенерировано после завершения транзакции, т. е. после отправки данных и получения возможных данных down или запрошенного подтверждения.

В отличие от `LMIC_setTxData()`, этот API не будет пытаться настроить скорость передачи данных, если сообщение слишком длинное для текущей скорости передачи данных. Полное обсуждение см. в разделе 2.5.13.

Этот API предпочтителен для новых приложений по двум причинам. Во-первых, автоматическое увеличение скорости передачи данных, возможно, не соответствует требованиям. Во-вторых, это не всегда возможно. Лучше, чтобы приложения были разработаны для управления работой в условиях низкой скорости передачи данных.

Из-за многочисленных постусловий, которые необходимо проверить после вызова `LMIC_setTxData_strict()`, мы настоятельно рекомендуем использовать вместо этого `LMIC_setTxData2_strict()` или `LMIC_sendWithCallback_strict()`.

2.5.15 `lmic_tx_error_t` `LMIC_setTxData2` (`u1_t` порт, `xref2u1_t` данные, `u1_t` `dlen`, `u1_t` подтвержденный)

Подготовьте передачу данных вверх по течению в следующее возможное время. Удобная функция для `LMIC_setTxData()`. Если данные равны NULL, будут использоваться данные в `LMIC.pendTxData[]`.

Для совместимости с существующими приложениями, если включена адаптация скорости передачи данных, эта функция проверит, возможно ли передать сообщение с текущей скоростью передачи данных. Если нет, скорость передачи данных будет увеличена, если это возможно. Полное обсуждение см. в разделе 2.5.13.

Тип результата `lmic_tx_error_t` является синонимом `int`. Могут быть возвращены следующие значения.

Имя	Значение	Описание
<code>LMIC_ERROR_SUCCESS</code>	0	Ошибок не произошло, будет опубликовано <code>EV_TXCOMPLETE</code> .
<code>LMIC_ERROR_TX_BUSY</code>	-1	LMIC был занят отправкой другого сообщения. Это сообщение не было отправлено. <code>EV_TXCOMPLETE</code> не будет опубликовано для этого сообщения.
<code>LMIC_ERROR_TX_TOO_LARGE</code>	-2	Сообщение в очереди слишком велико для любой скорости передачи данных для этого региона. Это сообщение не было отправлено. <code>EV_TXCOMPLETE</code> не будет опубликовано для этого сообщения.
<code>LMIC_ERROR_TX_NOT_FEASIBLE -3</code>		Сообщение в очереди невыполнимо для любой разрешенной скорости передачи данных. Это сообщение не было отправлено. <code>EV_TXCOMPLETE</code> не будет опубликовано для этого сообщения.
<code>LMIC_ERROR_TX_FAILED</code>	-4	Сообщение в очереди не удалось передать по какой-то причине, связанной с длиной данных, во время первоначального вызова LMIC для его передачи. Это сообщение не было отправлено. <code>EV_TXCOMPLETE</code> не будет опубликовано для этого сообщения.

2.5.16 `lmic_tx_error_t` `LMIC_setTxData2_strict` (`порт u1_t`, данные `xref2u1_t`, `dlen u1_t`, `u1_t` подтвержденный)

Подготовить передачу данных вверх по течению в следующее возможное время. Эта функция идентична `LMIC_setTxData2()`, за исключением того, что текущая скорость передачи данных никогда не будет изменена. Таким образом, возвращаемая ошибка `LMIC_ERROR_TX_NOT_FEASIBLE` имеет немного иное значение. Полное обсуждение см. в 2.5.13.

Имя	Значение	Описание
<code>LMIC_ERROR_TX_NOT_FEASIBLE -3</code>		Сообщение в очереди невыполнимо для текущей скорости передачи данных. Это сообщение не было отправлено. <code>EV_TXCOMPLETE</code> не будет опубликовано для этого сообщения.

2.5.17 `lmic_tx_error_t LMIC_sendWithCallback ( порт u1_t, данные u1_t, dlen u1_t, подтверждено u1_t, lmic_txmessage_cb_t *pCb, void *pUserData)`

Подготовьте передачу данных вверх по течению в следующее возможное время и вызовите указанную функцию, когда передача завершится. Удобная функция для `LMIC_setTxData()`. Если `data` равен `NULL`, будут использоваться данные в `LMIC.pendTxData[]`. Аргументы `dlen`, `confirmed` и `port` имеют то же значение, что и для `LMIC_setTxData2()`.

Если первоначальный вызов успешен, будет вызван обратный вызов, а также будет выдано событие `EV_TXCOMPLETE`. Полное обсуждение см. в разделе 2.5.21.

Для совместимости с существующими приложениями, если включена адаптация скорости передачи данных, эта функция проверит, возможно ли передать сообщение с текущей скоростью передачи данных. Если нет, скорость передачи данных будет увеличена, если это возможно. Полное обсуждение см. в разделе 2.5.13.

Функция обратного вызова имеет прототип:

```
typedef void (lmic_txmessage_cb_t)(void *pUserData, int fSuccess);
```

Он вызывается LMIC, когда передача сообщения завершена. `fSuccess` будет ненулевым для передачи, которая считается успешной, или нулевым, если передача считается неудавшейся. `pUserData` устанавливается на значение, переданное в соответствующем вызове `LMIC_sendWithCallback()`.

Список возможных кодов результата см. в разделе 2.5.15. Во всех случаях, если результат отличается от `LMIC_ERROR_SUCCESS`, функция обратного вызова пользователя не была вызвана и не будет вызвана, а `EV_TXCOMPLETE` не будет выведен.

2.5.18 `lmic_tx_error_t LMIC_sendWithCallback_strict (u1_t порт, u1_t *данные, u1_t dlen, u1_t подтверждено, lmic_txmessage_cb_t *pCb, void *pUserData)`

Подготовить передачу данных вверх по течению в следующее возможное время и вызвать указанную функцию, когда передача завершится. Эта функция идентична `LMIC_sendWithCallback()`, за исключением того, что она не будет пытаться изменить текущую скорость передачи данных, если текущая передача невозможна. (См. 2.5.13 для полного обсуждения.) Таким образом, возвращаемая ошибка `LMIC_ERROR_TX_NOT_FEASIBLE` имеет немного иное значение, как описано в разделе 2.5.16.

2.5.19 `недействительный LMIC_clrTxData ()`

Удалить данные, ранее подготовленные для передачи вверх по течению. Если операции `LMIC_sendWithCallback()` или `LMIC_sendWithCallback_strict()` ожидают выполнения, функция обратного вызова будет вызвана с `fSuccess`, установленным в ноль. Если сообщения передачи ожидают выполнения, будет сообщено о событии `EV_TXCOMPLETE`.

2.5.20 `void LMIC_registerRxMessageCb (lmic_rxmessage_cb_t *pRxMessageCb, void *pUserData)`

Эта функция регистрирует функцию обратного вызова, которая будет вызвана, когда LMIC обнаружит прием сообщение.

Функция обратного вызова имеет прототип:

```
typedef void (lmic_rxmessage_cb_t)(
    void *pUserData, u1_t порт, const u1_t *pMessage, size_t nMessage
);
```


Эта функция вызывается из LMIC; она должна избегать вызова API LMIC и операций, критичных по времени.

Аргумент `pUserData` устанавливается на значение, переданное в `LMIC_registerRxMessageCb()`. Аргумент `port` устанавливается на номер порта сообщения. Если ноль, то было получено сообщение MAC, и `nMessage` будет равен нулю. Аргумент `pMessage` устанавливается так, чтобы он указывал на первый байт сообщения во внутренних буферах LMIC, а `nMessage` устанавливается на количество байтов данных.

Можно выделить следующие состояния.

порт n	Message	Значение
0	0	Сообщение MAC получено на порту 0. Оно уже обработано, но доставлено клиенту для проверки. LMIC отклонит все сообщения с прицепленными данными MAC, нацеленными на порт 0.
0	0	Получена пустая полезная нагрузка (без порта, без кадра). Скорее всего, есть данные MAC-адреса. См. ниже.
0	0	Получена пустая полезная нагрузка для определенного порта. Интерпретировать это — задача приложения.
0	0	Получена непустая полезная нагрузка для определенного порта. Интерпретация этого зависит от приложения.

Если `port` равен нулю, а `nMessage` равен нулю, то можно обнаружить и проверить данные MAC-адреса, связанные с копированием, проверив значение `LMIC.dataBeg`. Если не равно нулю, то есть байты `LMIC.dataBeg` данных, связанных с копированием, и эти данные можно найти в `LMIC.frame[0]` через `LMIC.frame[LMIC.dataBeg-1]`.

Если порт не равен нулю, данные MAC-адреса, полученные с помощью Piggyback, также можно проверить с помощью значения `LMIC.dataBeg`. Если значение больше 1, то имеется (`LMIC.dataBeg-1`) байт данных, полученных с помощью Piggyback, и эти данные можно найти в `LMIC.frame[0]` через `LMIC.frame[LMIC.dataBeg-2]`.

Имейте в виду, что если вы также используете обратные вызовы событий, события также будут сообщаться функциям прослушивания событий. Полное обсуждение см. в разделе 2.5.21.

2.5.21 void LMIC_registerEventCb (lmic_event_cb_t *pEventCb, void *pUserData)

Эта функция регистрирует функцию обратного вызова, которая будет вызвана при обнаружении LMIC события.

Функция обратного вызова имеет прототип:

```
typedef void (lmic_event_cb_t)(void *pUserData, ev_t ev);
```

Аргумент `ev` устанавливается в числовой код, указывающий тип произошедшего события. Аргумент `pUserData` устанавливается в соответствии со значением, переданным в `LMIC_registerEventCb()`.

Реализация этой функции обратного вызова может реагировать на определенные события и запускать новые действия на основе события и состояния LMIC. Обычно реализация обрабатывает интересующие ее события и планирует дальнейшие действия протокола с использованием API LMIC. Будут сообщены следующие события:

- EV_JOINING
Узел начал подключаться к сети.
- EV_JOINED
Узел успешно подключился к сети и теперь готов к обмену данными.

- EV_JOIN_FAILED

Узел не смог подключиться к сети (после повторной попытки).

- EV_REJOIN_FAILED

Узел не присоединился к новой сети, но все еще может быть подключен к старой сети. Эта функция (попытка присоединиться к новой сети, будучи подключенным к старой) устарела и будет удалена в будущих версиях.

- EV_TXCOMPLETE

Данные, подготовленные с помощью LMIC_setTxData(), были отправлены, и окно приема для нисходящих данных завершено. Если было запрошено подтверждение, то оно получено. При обработке этого события код также может проверить прием данных. См. 2.5.21.1

для получения подробной информации. Если вы используете LMIC_registerRxMessageCb(), лучше оставить это LMIC, который вызовет функцию получения сообщения клиента по мере необходимости.

- EV_RXCOMPLETE

Получен нисходящий канал связи либо в слоте RX класса A, либо в слоте ring класса B, либо (в будущем) в окне приема класса C. Код должен проверить полученные данные. Подробности см. в разделе 2.5.21.1. При использовании LMIC_sendWithCallback() и LMIC_registerRxMessageCb() игнорируйте EV_RXCOMPLETE в функции обработки событий.

- EV_SCAN_TIMEOUT

После вызова LMIC_enableTracking() в течение интервала маяка не было получено ни одного сигнала.

Необходимо перезапустить отслеживание.

- EV_BEACON_FOUND

После вызова LMIC_enableTracking() был получен первый маяк в пределах интервала маяка.

- EV_BEACON_TRACKED

Следующий маяк был получен в ожидаемое время.

- EV_BEACON_MISSED

В ожидаемое время сигнал маяка не был получен.

- EV_LOST_TSYNC

Веасон был пропущен повторно, и синхронизация времени была потеряна. Трекинг или пингование необходимо перезапустить.

- EV_RESET

Сессия сброшена из-за сброса счетчиков последовательностей. Сеть будет автоматически переподключена для получения новой сессии.

- EV_LINK_DEAD

Подтверждение от сетевого сервера не поступало в течение длительного периода времени.

Передачи все еще возможны, но их прием ненадежен.

- EV_LINK_ALIVE

Ссылка была мертва, но теперь снова жива.

- EV_SCAN_FOUND

Это событие зарезервировано для будущего использования и никогда не сообщается.

- EV_TXSTART

Это событие сообщается непосредственно перед тем, как водителю радиостанции дается команда начать передачу.

- EV_TXCANCELED

Ожидаемая передача была отменена либо из-за запроса на отмену, либо как побочный эффект запроса API, либо как побочный эффект изменения на уровне MAC (например, переполнения счетчика кадров).

- EV_RXSTART

LMIC собирается открыть окно приема. Очень важно, чтобы процедура обработки событий выполняла как можно меньше работы – не более одной миллисекунды реального времени, в противном случае нисходящие каналы могут работать неправильно. Не печатайте ничего во время обработки этого события; сохраните данные для печати позже. Это событие не отправляется в `onEvent()` подпрограмма, раздел 2.3.4; отправляется только обработчикам событий, зарегистрированным через `LMIC_registerEventCb()`, раздел 2.5.21.

- EV_JOIN_TXCOMPLETE

Это событие указывает на конец цикла передачи для JOIN. Оно указывает на то, что оба окна приема `join` были обработаны без получения сообщения `JoinAccept` от сети.

Информацию о состоянии LMIC для конкретных событий можно получить из глобальной структуры LMIC, описанной в разделе 2.4.

Функции событий и функции обратного вызова передачи/приема являются ортогональными. Для данного события могут быть вызваны несколько процедур.

Последовательность следующая.

1. При использовании функции `onEvent()` (LMIC настроен на использование `onEvent`, а функция с именем `onEvent` предоставляется при использовании компиляторов, поддерживающих слабые ссылки), вызывается функция `onEvent()`. (Для совместимости событие EV_RXSTART никогда не отправляется в `onEvent()` функция.)
2. Если событие указывает на то, что сообщение было получено, и зарегистрирован обратный вызов получения, вызывается обратный вызов приема.
3. Если событие указывает на то, что передача завершена, и сообщение было отправлено с одним из API обратного вызова вызывается обратный вызов клиента.
4. Наконец, если клиент зарегистрировал обратный вызов события, вызывается зарегистрированный обратный вызов.

2.5.21.1 Получение данных по нисходящей линии связи с помощью функции события

При получении EV_TXCOMPLETE или EV_RXCOMPLETE код обработки событий должен проверить наличие данных нисходящей линии связи и передать их приложению. Для этого используйте следующий код.

```
// Какие-нибудь данные должны быть получены?
если (LMIC.dataLen != 0) {
    // Данные получены. Извлечь номер порта, если есть.
    u1_t bPort = 0;
    если (LMIC.txrxFlags & TXRX_PORT)
        bPort = LMIC.frame[LMIC.dataBeg - 1];
    // Вызов пользовательской функции с номером порта, pMessage, nMessage
    receiveMessage(
        bPort, LMIC.frame + LMIC.dataBeg, LMIC.dataLen
    );
}
```

Если вы хотите поддерживать оповещение клиента о сообщениях нулевой длины, необходимо использовать немного более сложный код.

```
// Какие-нибудь данные должны быть получены?
if (LMIC.dataLen != 0 || LMIC.dataBeg != 0) {
```

```

// Данные получены. Извлечь номер порта, если есть.
u1_t bPort = 0;
если (LMIC.txrxFlags & TXRX_PORT)
    bPort = LMIC.frame[LMIC.dataBeg - 1];
// Вызов пользовательской функции с номером порта, pMessage, nMessage;
// nMessage может быть равен нулю.
receiveMessage(
    bPort, LMIC.frame + LMIC.dataBeg, LMIC.dataLen
);
}

```

2.5.22 bit_t LMIC_enableTracking (u1_t tryBcnInfo)

Включить отслеживание маяков. Значение 0 для tryBcnInfo указывает на немедленное начало сканирования маяков. Ненулевое значение указывает количество попыток запроса сервера для точного времени прибытия маяков. Запросы будут отправлены в следующих кадрах восходящего потока (кадр не будет сгенерирован). Если ответ не получен, будет запущено сканирование. События EV_BEACON_FOUND или EV_SCAN_TIMEOUT будут сгенерированы для первого маяка, а события EV_BEACON_TRACKED, EV_BEACON_MISSED или EV_LOST_TSYNC будут сгенерированы для последующих маяков.

2.5.23 недействительный LMIC_disableTracking ()

Отключить отслеживание маяка. Маяк больше не будет отслеживаться, и, следовательно, пингование также будет отключено.

2.5.24 void LMIC_setPingable (u1_t intvExp)

Включить пинг и установить интервал прослушивания нисходящего потока. Пинг будет включен со следующим восходящим потоком кадр (кадр не будет сгенерирован). Интервал прослушивания составляет 2^{intvExp} секунд, допустимые значения для intvExp 0-7. Эта функция API требует действительного сеанса, установленного с сетевым сервером либо через функции LMIC_startJoining(), либо LMIC_setSession() (см. разделы 2.5.2 и 2.5.4). Если отслеживание маяка еще не включено, сканирование будет запущено немедленно. Чтобы избежать сканирования, маяк можно найти более эффективно, выполнив предшествующий вызов LMIC_enableTracking() с ненулевым параметром. В дополнение к событиям, упомянутым для LMIC_enableTracking(), событие EV_RXCOMPLETE будет генерироваться всякий раз, когда в слоте ping будут получены данные нисходящего потока.

2.5.25 недействительный LMIC_stopPingable ()

Остановить прослушивание нисходящих данных. Периодический прием отключен, но маяки все равно будут отслеживаться. Чтобы остановить отслеживание, маяку требуется вызов LMIC_disableTracking().

2.5.26 недействительный LMIC_sendAlive ()

Отправьте один пустой восходящий MAC-кадр как можно скорее. Может использоваться для сигнализации жизнеспособности или для передачи ожидающих MAC-опций, а также для открытия окна приема.

2.5.27 недействительный LMIC_shutdown ()

Остановить всю активность MAC. После этого MAC необходимо сбросить с помощью вызова LMIC_reset() и инициировать новые действия протокола.

2.5.28 void LMIC_requestNetworkTime (lmic_request_network_time_cb_t *, void *pUserData)

Зарегистрировать запрос MAC сетевого времени для пересылки со следующим кадром восходящей линии связи. Первый аргумент — указатель на функцию со следующей сигнатурой.

```
typedef void LMIC_ABI_STD lmic_request_network_time_cb_t (void * pUserData, int flagSuccess);
```

Эта функция вызывается после завершения обработки. flagSuccess будет ненулевым, если время было успешно получено, и нулевым в противном случае. pUserData в обратном вызове будет установлен в соответствии со значением pUserData в исходном вызове LMIC_requestNetworkTime(). Если не используется, используйте указатель NULL.

2.5.29 int LMIC_getNetworkTimeReference (lmic_time_reference_t *pReference)

Извлечь ссылку на время для последнего полученного сетевого времени.

Структура в pReference обновляется соответствующей информацией. Результатом этого вызова является булево значение указывает, было ли возвращено допустимое время, если ненулевое, было возвращено допустимое время, в противном случае допустимое время не было доступно (или pReference было NULL). В случае неудачи *pReference не изменяется.

Структура lmic_time_reference_t имеет следующие поля.

```
typedef структура {
    ostime_t tLocal;
    lmic_gpstime_t tNetwork;
} lmic_time_reference_t;
```

tNetwork устанавливается на время GPS, переданное сетью в сообщении DeviceTimeAns. tLocal вычисляется путем преобразования дробной части сообщения DeviceTimeAns в такты ОС и вычитания этого значения из времени завершения сообщения DeviceTimeReq.

Два поля устанавливают связь между заданным временем ОС tLocal и заданным временем GPS tNetwork. Из этого можно вычислить текущее время ОС, соответствующее заданному времени GPS, используя формулу типа $ostime = ref.tLocal + sec2osticks(gpstime - ref.tNetwork)$.

2.5.30 uint8_t LMIC_setBatteryLevel (uint8_t uBattLevel)

Установите уровень заряда батареи, который будет возвращаться сетевому серверу MAC в сообщениях DevStatusAns. Результатом этого вызова является предыдущее значение. Внутреннее значение инициализируется LMIC_init() до MCMD_DEVS_NOINFO. Внутреннее значение не изменяется LMIC_reset().

Возможные значения uBattLevel:

Имя значения	Значение
0 MCMD_DEVS_EXT_POWER	Устройство работает от внешнего источника питания.
0x01 MCMD_DEVS_BATT_MIN	Устройство работает от аккумулятора; уровень заряда аккумулятора минимальный.
...	
0xFE MCMD_DEVS_BATT_MAX	Устройство работает от батареи; батарея заряжена на максимальном уровне
0xFF MCMD_DEVS_BATT_NOINFO	Устройство не знает состояние своей батареи/питания и не смогло измерить уровень заряда батареи.

Если приложению известен уровень заряда батареи в процентах от емкости (от 0% до 100% включительно), оно должно выполнить расчет следующим образом.

```
uBattLevel = (MCMD_DEVS_BAT_MAX - MCMD_DEVS_BAT_MIN253 + 50) *      uПроцент заряда батареи / 100;  
uBattLevel += MCMD_DEVS_BAT_MIN;  
  
LMIC_setBatteryLevel(uBattLevel);
```

2.5.31 uint8_t LMIC_getBatteryLevel (пусто)

Эта функция возвращает текущий уровень заряда батареи, сохраненный для использования MAC. Значение соответствует описанию в LMIC_setBatteryLevel().

3. Уровень абстракции оборудования

Библиотека LMIC разделена на большую часть переносимого кода и небольшую часть, специфичную для платформы. Реализуя функции этого слоя аппаратной абстракции с указанной семантикой, библиотека может быть легко перенесена на новые аппаратные платформы.

3.1 Интерфейс HAL

Должны поддерживаться следующие группы аппаратных компонентов:

- Для управления антенным переключателем радиоприемника (RX) в режиме вывода требуется до четырех цифровых линий ввода/вывода. и TX), выбор микросхемы SPI (NSS) и линия сброса (RST).
- Для обнаружения передатчика и приемника радиостанции в режиме ввода необходимы три цифровые линии ввода/вывода. состояния (DIO0, DIO1 и DIO2).
- Для чтения и записи регистров радиоприемника необходим модуль SPI.
- Для точной регистрации событий и планирования новых протокольных действий необходим таймер.
- Контроллер прерываний может использоваться для пересылки прерываний, генерируемых цифровыми входными линиями.

В этом разделе описывается функциональный интерфейс, необходимый для доступа к этим аппаратным компонентам:

3.1.1 недействительный `lmic_hal_init()`

Инициализируйте уровень абстракции оборудования. Настройте все компоненты (IO, SPI, TIMER, IRQ) для дальнейшего использования с функциями `lmic_hal_xxx()`. Эта функция устарела. Библиотека LMIC вместо этого вызывает `lmic_hal_init_ex()`. Клиент не может вызвать `lmic_hal_init()` или `lmic_hal_init_ex()` напрямую, так как они вызываются из `os_init()/os_init_ex()`, и их необходимо вызывать только один раз.

3.1.2 `void lmic_hal_init_ex(const void *pHalData)`

Инициализируйте уровень абстракции оборудования. Настройте все компоненты (IO, SPI, TIMER, IRQ) для дальнейшего использования с функциями `lmic_hal_xxx()`. `pHalData` — это указатель на данные, специфичные для HAL. При работе с Arduino HAL это должен быть указатель на структуру `lmic_pinmap`. Библиотека LMIC вызывает `lmic_hal_init_ex()`. Клиент не может вызвать `lmic_hal_init()` или `lmic_hal_init_ex()` напрямую, так как они вызываются из `os_init()/os_init_ex()` и должны быть вызваны только один раз.

3.1.3 недействительный `lmic_hal_failed()`

Выполнить действие «фатального сбоя». Эта функция будет вызвана утверждениями кода при фатальных условиях. Возможные действия: HALT или перезагрузка.

3.1.4 `void lmic_hal_pin_rxtx (значение u1_t)`

Управляйте цифровыми выходными контактами RX и TX (0=прием, 1=передача).

3.1.5 `void lmic_hal_pin_rst (значение u1_t)`

Управление выводом RST радио (0=низкий, 1=высокий, 2=плавающий)

3.1.6 void radio_irq_handler (u1_t dio)

Когда HAL обнаруживает нарастающий фронт на любой из трех входных линий DIO0, DIO1 и DIO2, он должен уведомить LMIC. Он может сделать это, вызвав функцию `radio_irq_handler()`. Он должен установить `dio` для указания линии, которая сгенерировала прерывание (0, 1, 2). Эта процедура является оболочкой для `radio_irq_handler_v2()` и просто вызывает `os_getTime()` для получения текущего времени. Если ваше оборудование может более точно захватывать время прерывания, ваш HAL должен использовать `radio_irq_handler_v2()`.

3.1.7 void radio_irq_handler_v2 (u1_t dio, os_time_t tIrq)

Когда HAL обнаруживает нарастающий фронт на любой из трех входных линий DIO0, DIO1 и DIO2, он должен уведомить LMIC. Если у HAL есть высокоточная временная метка для момента изменения состояния линии, он должен вызвать функцию `radio_irq_handler_v2()`. Установите `dio` для указания измененной линии (0, 1, 2). Установите `tIrq` к временной метке изменения состояния линии.

3.1.8 void lmic_hal_spi_read(u1_t cmd, u1_t* buf, size_t len)

Выполнить чтение SPI. Записать командный байт `cmd`, затем прочитать `len` байт в буфер, начиная с `buf`.

3.1.9 void lmic_hal_spi_write(u1_t cmd, const u1_t* buf, size_t len)

Выполнить запись SPI. Записать командный байт `cmd`, а затем `len` байт из буфера, начиная с `buf`.

3.1.10 u4_t lmic_hal_ticks ()

Возвращает 32-битное системное время в тиках (те же единицы, что и `ostime_t`), но обратите внимание, что это беззнаковое время, тогда как `ostime_t` имеет знак.

3.1.11 void lmic_hal_waitUntil (время u4_t)

Ожидание в режиме занятости до достижения указанной временной метки (в тиках).

3.1.12 u1_t lmic_hal_checkTimer (u4_t targettime)

Проверить и перемотать таймер на заданное `targettime`. Возвратить 1, если `targettime` близко (не стоит программировать таймер). В противном случае перемотать таймер на точное `targettime` или на полный период таймера и вернуть 0. Единственное действие, которое требуется при достижении `targettime`, — это пробуждение ЦП из возможных состояний сна.

3.1.13 недействительный lmic_hal_disableIRQs ()

Отключить все прерывания ЦП. Может вызываться вложенно. Но всегда будет сопровождаться соответствующим вызовом `lmic_hal_enableIRQs()`.

3.1.14 void lmic_hal_enableIRQs ()

Включить прерывания ЦП. При вложенном вызове только самый внешний вызов должен фактически включить прерывания.

3.1.15 недействительный lmic_hal_sleep ()

Спать до тех пор, пока не произойдет прерывание. Предпочтительно, чтобы системные компоненты были переведены в режим пониженного энергопотребления перед сном и повторно инициализированы после сна. При использовании эталонной реализации Arduino это не требуется; LMIC возвращается к вызывающей стороне, которая отвечает за организацию сна.

3.1.16 `sl_t_lmic_hal_getRssiCal ()`

Получите калибровку RSSI для радио в дБ. Драйвер радио добавляет это к указанному RSSI для преобразования в абсолютный дБ для выполнения вычислений listen-before-talk. Не используется, если listen-before-talk не настроен для этого региона.

3.1.17 `ostime_t_lmic_hal_setModulePower (логическое значение)`

Запрос на включение или выключение питания модуля. Если true, питание TCXO должно быть активировано, и должны быть активированы любые обычно высоко-Z линии управления. Эта функция возвращает количество тактов задержки, которые должны быть вставлены перед использованием радио. Если управление питанием на уровне модуля не реализовано или если радио уже находится в желаемом состоянии, эта процедура может просто вернуть ноль. Обычно при включении питания требуется задержка в несколько миллисекунд, но задержка не требуется, если питание уже включено или при выключении питания.

3.2 Реализация эталонного HAL для Arduino

Библиотека Arduino LMIC включает в себя эталонную реализацию HAL для Arduino. Пожалуйста, обратитесь к README.md для получения информации о реализации.

4. Примеры

Приведен ряд примеров, демонстрирующих, как можно реализовать типичные узловые приложения с помощью всего лишь нескольких строк кода с использованием библиотеки LMIC.

5. История релизов

История выпусков Arduino LMIC находится в файле README.md по адресу <https://github.com/mcci-catena/arduino-lmic>.

5.1 История выпусков IBM

Версия и дата	Описание
Версия 1.0 Ноябрь 2014 г.	Первоначальная версия.
Версия 1.1 Январь 2015 г.	Добавлен API LMIC_setSession(). Незначительные внутренние исправления.
Версия 1.2 Февраль 2015 г.	Добавлены API LMIC_setupBand(), LMIC_setupChannel(), LMIC_disableChannel(), LMIC_setLinkCheckMode(). Незначительные внутренние исправления.
В 1.4 Март 2015 г.	Измененный API: флаг индикатора порта в LMIC.txrxFlags был инвертирован (теперь TXRX_PORT, ранее TXRX_NOPORT). Внутренние исправления ошибок. Форматирование документа.
В 1.5 Май 2015 г.	Исправления ошибок и обновление документации.
В 1.6 Июль 2016	Изменена лицензия на BSD. Включено приложение модема (см. examples/modem и LMIC-Modem.pdf). Добавлены драйверы оборудования STM32 и периферийный код, специфичный для платы Blipreg.